

BOOK 8 — COMPLETE CODING PRACTICE & INTERVIEW PREPARATION

Complete Fresher / Entry-Level Java Full Stack Preparation Series

BOOK 8 — COMPLETE CODING PRACTICE & INTERVIEW PREPARATION

Complete Fresher / Entry-Level Java Full Stack Preparation Series

CHAPTER 1 — CORE LOGIC & ALGORITHMIC CODING PRACTICE

1.1 Number-Based Coding Practice (Syllabus 26.1.1 to 26.1.8)

Problem 1: Prime Number Check (26.1.1)

```
import java.util.Scanner;

public class PrimeCheckDemo {
    // Method to check if a number is prime
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        // Check divisors up to square root of n for efficiency
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) return false; // Found a factor, not prime
        }
        return true;
    }

    public static void main(String[] args) {
        int number = 29;
        if (isPrime(number)) {
            System.out.println(number + " is a Prime Number.");
        } else {
            System.out.println(number + " is NOT a Prime Number.");
        }
    }
}
```

- **Expected Output:** 29 is a Prime Number.

- **Execution Explanation:** A prime number is a natural number greater than 1 with no positive divisors other than 1 and itself. Checking factors up to $\sqrt{n}$ optimizes time complexity to $O(\sqrt{n})$.

Problem 2: Factorial Calculation (26.1.2)

```

public class FactorialDemo {
    // Iterative method for factorial
    public static long calculateFactorial(int n) {
        if (n < 0) return -1; // Invalid for negative numbers
        long result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i; // Accumulate product
        }
        return result;
    }

    public static void main(String[] args) {
        int num = 5;
        System.out.println("Factorial of " + num + " = " + calculateFactorial(num));
    }
}

```

• **Expected Output:** `Factorial of 5 = 120`

• **Execution Explanation:** Factorial $n! = 1 \times 2 \times \dots \times n$. Uses `long` primitive to avoid integer overflow for values up to 20.

Problem 3: Fibonacci Series (26.1.3)

```

public class FibonacciDemo {
    public static void printFibonacci(int count) {
        int a = 0, b = 1;
        System.out.print("Fibonacci Series (" + count + " terms): " + a + " " + b);
        for (int i = 2; i < count; i++) {
            int next = a + b;
            System.out.print(" " + next);
            a = b;
            b = next;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        printFibonacci(8);
    }
}

```

• **Expected Output:** `Fibonacci Series (8 terms): 0 1 1 2 3 5 8 13`

• **Execution Explanation:** Each term is the sum of the preceding two terms ($F_n = F_{n-1} + F_{n-2}$).

Problem 4: Reverse a Number (26.1.4)

```

public class ReverseNumberDemo {
    public static int reverse(int number) {
        int reversed = 0;
        while (number != 0) {
            int digit = number % 10; // Extract last digit
            reversed = reversed * 10 + digit; // Append digit
            number /= 10; // Remove last digit
        }
        return reversed;
    }

    public static void main(String[] args) {
        int num = 12345;
        System.out.println("Original: " + num + " | Reversed: " + reverse(num));
    }
}

```

- **Expected Output:** Original: 12345 | Reversed: 54321
- **Execution Explanation:** Modulo % 10 extracts the rightmost digit, and division / 10 strips it until the number becomes zero.

Problem 5: Palindrome Number Check (26.1.5)

```

public class PalindromeNumberDemo {
    public static boolean isPalindrome(int number) {
        if (number < 0) return false;
        int original = number;
        int reversed = 0;
        while (number > 0) {
            reversed = reversed * 10 + (number % 10);
            number /= 10;
        }
        return original == reversed;
    }

    public static void main(String[] args) {
        int num = 12321;
        System.out.println(num + " is Palindrome? " + isPalindrome(num));
    }
}

```

- **Expected Output:** 12321 is Palindrome? true
- **Execution Explanation:** Reverses the digits and compares the result with the original value.

Problem 6: Armstrong Number Check (26.1.6)

```

public class ArmstrongNumberDemo {
    public static boolean isArmstrong(int number) {
        int original = number;
        int digits = String.valueOf(number).length();
        int sum = 0;
        int temp = number;

        while (temp > 0) {
            int digit = temp % 10;
            sum += Math.pow(digit, digits); // Raise digit to total count power
            temp /= 10;
        }
        return sum == original;
    }

    public static void main(String[] args) {
        int num = 153; // 1^3 + 5^3 + 3^3 = 153
        System.out.println(num + " is Armstrong? " + isArmstrong(num));
    }
}

```

- **Expected Output:** 153 is Armstrong? true

- **Execution Explanation:** An N -digit number is an Armstrong number if the sum of its digits raised to the N -th power equals the original number.

Problem 7: Perfect Number Check (26.1.7)

```

public class PerfectNumberDemo {
    public static boolean isPerfect(int number) {
        if (number <= 1) return false;
        int sum = 1; // 1 is always a proper divisor
        for (int i = 2; i <= number / 2; i++) {
            if (number % i == 0) sum += i;
        }
        return sum == number;
    }

    public static void main(String[] args) {
        int num = 28; // Divisors: 1 + 2 + 4 + 7 + 14 = 28
        System.out.println(num + " is Perfect? " + isPerfect(num));
    }
}

```

- **Expected Output:** 28 is Perfect? true

- **Execution Explanation:** A perfect number is a positive integer equal to the sum of its positive proper divisors.

Problem 8: Strong Number Check (26.1.8)

```

public class StrongNumberDemo {
    private static int factorial(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++) fact *= i;
        return fact;
    }

    public static boolean isStrong(int number) {
        int original = number;
        int sum = 0;
        int temp = number;
        while (temp > 0) {
            int digit = temp % 10;
            sum += factorial(digit); // Sum factorials of individual digits
            temp /= 10;
        }
        return sum == original;
    }

    public static void main(String[] args) {
        int num = 145; // 1! + 4! + 5! = 1 + 24 + 120 = 145
        System.out.println(num + " is Strong? " + isStrong(num));
    }
}

```

- **Expected Output:** 145 is Strong? true
- **Execution Explanation:** A number is strong if the sum of the factorials of its digits equals the original number.

1.2 String-Based Coding Practice (Syllabus 26.2.1 to 26.2.6)

Problem 1: Reverse String (26.2.1)

```

public class ReverseStringDemo {
    public static String reverse(String str) {
        char[] chars = str.toCharArray();
        int left = 0, right = chars.length - 1;
        while (left < right) {
            char temp = chars[left];
            chars[left] = chars[right];
            chars[right] = temp;
            left++;
            right--;
        }
        return new String(chars);
    }

    public static void main(String[] args) {
        String str = "FullStack";
        System.out.println("Original: " + str + " | Reversed: " + reverse(str));
    }
}

```

- **Expected Output:** Original: FullStack | Reversed: kcatSlluF
- **Execution Explanation:** Uses a two-pointer technique to swap characters in-place inside a `char[]` array in $O(N)$ time.

Problem 2: Check Palindrome String (26.2.2)

```
public class PalindromeStringDemo {
    public static boolean isPalindrome(String str) {
        String cleaned = str.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
        int left = 0, right = cleaned.length() - 1;
        while (left < right) {
            if (cleaned.charAt(left) != cleaned.charAt(right)) return false;
            left++;
            right--;
        }
        return true;
    }

    public static void main(String[] args) {
        String input = "A man, a plan, a canal: Panama";
        System.out.println("Is Palindrome? " + isPalindrome(input));
    }
}
```

- **Expected Output:** `Is Palindrome? true`

- **Execution Explanation:** Strips non-alphanumeric characters and checks character symmetry using two pointers.

Problem 3: Anagram Check (26.2.3)

```
import java.util.Arrays;

public class AnagramCheckDemo {
    public static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
        char[] a1 = s1.toLowerCase().toCharArray();
        char[] a2 = s2.toLowerCase().toCharArray();
        Arrays.sort(a1);
        Arrays.sort(a2);
        return Arrays.equals(a1, a2);
    }

    public static void main(String[] args) {
        String str1 = "Listen", str2 = "Silent";
        System.out.println("Are Anagrams? " + isAnagram(str1, str2));
    }
}
```

- **Expected Output:** `Are Anagrams? true`

- **Execution Explanation:** Two strings are anagrams if sorting their lowercase character arrays produces identical character sequences.

Problem 4: Character Frequency (26.2.4)

```
import java.util.HashMap;
import java.util.Map;

public class CharacterFrequencyDemo {
    public static void printFrequency(String str) {
        Map<Character, Integer> freqMap = new HashMap<>();
        for (char ch : str.toCharArray()) {
            freqMap.put(ch, freqMap.getOrDefault(ch, 0) + 1);
        }
        System.out.println("Character Frequencies: " + freqMap);
    }

    public static void main(String[] args) {
        printFrequency("javafullstack");
    }
}
```

- **Expected Output:** Character Frequencies: {a=3, c=1, s=1, t=1, u=1, f=1, v=1, j=1, l=2, k=2}
- **Execution Explanation:** Uses a [HashMap](#) to count and store occurrences of each character in $O(N)$ linear time.

Problem 5: Find Duplicate Characters (26.2.5)

```
import java.util.HashMap;
import java.util.Map;

public class DuplicateCharactersDemo {
    public static void findDuplicates(String str) {
        Map<Character, Integer> freqMap = new HashMap<>();
        for (char ch : str.toCharArray()) {
            freqMap.put(ch, freqMap.getOrDefault(ch, 0) + 1);
        }
        System.out.print("Duplicate Characters: ");
        for (Map.Entry<Character, Integer> entry : freqMap.entrySet()) {
            if (entry.getValue() > 1) {
                System.out.print(entry.getKey() + " ");
            }
        }
        System.out.println();
    }

    public static void main(String[] args) {
        findDuplicates("programming");
    }
}
```

- **Expected Output:** Duplicate Characters: r g m
- **Execution Explanation:** Filters map entries to display characters with frequency count greater than 1.

Problem 6: First Non-Repeated Character (26.2.6)

```
import java.util.LinkedHashMap;
import java.util.Map;

public class FirstNonRepeatedCharDemo {
    public static Character getFirstNonRepeated(String str) {
        Map<Character, Integer> freqMap = new LinkedHashMap<>(); // Maintains insertion order
        for (char ch : str.toCharArray()) {
            freqMap.put(ch, freqMap.getOrDefault(ch, 0) + 1);
        }
        for (Map.Entry<Character, Integer> entry : freqMap.entrySet()) {
            if (entry.getValue() == 1) return entry.getKey();
        }
        return null;
    }

    public static void main(String[] args) {
        String str = "swiss";
        System.out.println("First non-repeated char in 'swiss': " + getFirstNonRepeated(str));
    }
}
```

- **Expected Output:** First non-repeated char in 'swiss': w
- **Execution Explanation:** [LinkedHashMap](#) preserves character insertion order so the first key with count 1 is returned immediately.

1.3 Array-Based Coding Practice (Syllabus 26.3.1 to 26.3.6)

Problem 1: Find Largest / Smallest Element (26.3.1)

```
public class MinMaxArrayDemo {
    public static void findMinMax(int[] arr) {
        if (arr == null || arr.length == 0) return;
        int min = arr[0], max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) min = arr[i];
            if (arr[i] > max) max = arr[i];
        }
        System.out.println("Min: " + min + " | Max: " + max);
    }

    public static void main(String[] args) {
        int[] numbers = {45, 12, 89, 3, 67, 23};
        findMinMax(numbers);
    }
}
```

- **Expected Output:** Min: 3 | Max: 89
- **Execution Explanation:** Single-pass linear scan ($O(N)$) tracking running minimum and maximum values.

Problem 2: Find Second Largest Element (26.3.2)

```

public class SecondLargestDemo {
    public static int getSecondLargest(int[] arr) {
        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;

        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            } else if (num > secondLargest && num != largest) {
                secondLargest = num;
            }
        }
        return secondLargest;
    }

    public static void main(String[] args) {
        int[] arr = {12, 35, 1, 10, 34, 1};
        System.out.println("Second Largest: " + getSecondLargest(arr));
    }
}

```

- **Expected Output:** `Second Largest: 34`

- **Execution Explanation:** Updates `secondLargest` whenever a new maximum is found, handling duplicate maximum values correctly.

Problem 3: Find Duplicate Elements in Array (26.3.3)

```

import java.util.HashSet;
import java.util.Set;

public class ArrayDuplicatesDemo {
    public static void findDuplicates(int[] arr) {
        Set<Integer> seen = new HashSet<>();
        Set<Integer> duplicates = new HashSet<>();
        for (int num : arr) {
            if (!seen.add(num)) { // add() returns false if element already exists in set
                duplicates.add(num);
            }
        }
        System.out.println("Duplicates: " + duplicates);
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 2, 4, 5, 3, 6};
        findDuplicates(arr);
    }
}

```

- **Expected Output:** `Duplicates: [2, 3]`

- **Execution Explanation:** `HashSet.add()` returns `false` when an element is already present, identifying duplicate elements in $O(N)$ time.

Problem 4: Find Missing Number in $1^{\$}$ to $N^{\$}$ Array (26.3.4)

```

public class MissingNumberDemo {
    public static int findMissing(int[] arr, int n) {
        int expectedSum = n * (n + 1) / 2; // Mathematical sum formula
        int actualSum = 0;
        for (int num : arr) actualSum += num;
        return expectedSum - actualSum;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 4, 5, 6}; // N = 6, missing 3
        System.out.println("Missing Number: " + findMissing(arr, 6));
    }
}

```

- **Expected Output:** `Missing Number: 3`
- **Execution Explanation:** Subtracts actual array sum from Gauss's total sum formula $\frac{N(N+1)}{2}$.

Problem 5: Array Sorting (Bubble Sort) (26.3.5)

```

import java.util.Arrays;

public class BubbleSortDemo {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }
            if (!swapped) break; // Optimization: stop if array is already sorted
        }
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        bubbleSort(arr);
        System.out.println("Sorted Array: " + Arrays.toString(arr));
    }
}

```

- **Expected Output:** `Sorted Array: [11, 12, 22, 25, 34, 64, 90]`
- **Execution Explanation:** Adjacent element comparisons bubble the largest unsorted element to its correct end position in each pass.

Problem 6: Left & Right Array Rotation (26.3.6)

```

import java.util.Arrays;

public class ArrayRotationDemo {
    // Left rotate array by k positions
    public static void leftRotate(int[] arr, int k) {
        int n = arr.length;
        k = k % n;
        reverse(arr, 0, k - 1);
        reverse(arr, k, n - 1);
        reverse(arr, 0, n - 1);
    }

    private static void reverse(int[] arr, int start, int end) {
        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        leftRotate(arr, 2);
        System.out.println("Left Rotated by 2: " + Arrays.toString(arr));
    }
}

```

- **Expected Output:** Left Rotated by 2: [3, 4, 5, 1, 2]

- **Execution Explanation:** Reversing sub-arrays before reversing the full array rotates elements in $O(N)$ time with $O(1)$ auxiliary space.

CHAPTER 2 — ADVANCED DATA STRUCTURES, OOP & EXCEPTION CODING PRACTICE

2.1 Collection-Based Coding Practice (Syllabus 26.4.1 to 26.4.5)

Problem 1: ArrayList Manipulation (26.4.1)

```
import java.util.ArrayList;
import java.util.List;

public class ArrayListPracticeDemo {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Spring");
        list.add("Hibernate");

        // Insert at index 1
        list.add(1, "FullStack");
        // Remove element
        list.remove("Hibernate");

        System.out.println("ArrayList Elements: " + list);
        System.out.println("Contains 'Spring'? " + list.contains("Spring"));
    }
}
```

• **Expected Output:**

```
ArrayList Elements: [Java, FullStack, Spring]
Contains 'Spring'? true
```

- **Execution Explanation:** Demonstrates positional `add(index, element)`, `remove(Object)`, and `contains()` operations on dynamic arrays.

Problem 2: HashSet Duplicate Removal (26.4.2)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

public class HashSetDeduplicationDemo {
    public static void main(String[] args) {
        List<Integer> duplicatesList = Arrays.asList(10, 20, 10, 30, 20, 40, 50, 30);
        // Using LinkedHashSet to remove duplicates while preserving insertion order
        Set<Integer> uniqueSet = new LinkedHashSet<>(duplicatesList);

        List<Integer> uniqueList = new ArrayList<>(uniqueSet);
        System.out.println("Original: " + duplicatesList);
        System.out.println("Deduplicated: " + uniqueList);
    }
}
```

• **Expected Output:**

```
Original: [10, 20, 10, 30, 20, 40, 50, 30]
Deduplicated: [10, 20, 30, 40, 50]
```

- **Execution Explanation:** Passing a collection into a `Set` constructor filters duplicate elements based on `equals()` and `hashCode()`.

Problem 3: HashMap Frequency Counting (26.4.3)

```
import java.util.HashMap;
import java.util.Map;

public class HashMapFrequencyDemo {
    public static void main(String[] args) {
        String[] words = {"java", "spring", "java", "fullstack", "spring", "java"};
        Map<String, Integer> wordCounts = new HashMap<>();

        for (String word : words) {
            wordCounts.put(word, wordCounts.getOrDefault(word, 0) + 1);
        }

        System.out.println("Word Frequencies: " + wordCounts);
    }
}
```

- **Expected Output:** Word Frequencies: {spring=2, fullstack=1, java=3}
- **Execution Explanation:** `getOrDefault(key, 0)` simplifies frequency counter increments across array elements.

Problem 4: Custom Sorting with Comparable (26.4.4)

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

class Student implements Comparable<Student> {
    int id;
    String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Natural ordering by student ID
    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.id, other.id);
    }

    @Override
    public String toString() {
        return "Student{" + "id=" + id + ", name='" + name + '\'' + '}';
    }
}

public class ComparablePracticeDemo {
    public static void main(String[] args) {
        List<Student> students = new ArrayList<>();
        students.add(new Student(103, "Bob"));
        students.add(new Student(101, "Alice"));
        students.add(new Student(102, "Charlie"));

        Collections.sort(students); // Triggers compareTo()
        System.out.println("Sorted Students by ID: " + students);
    }
}

```

- **Expected Output:** Sorted Students by ID: [Student{id=101, name='Alice'}, Student{id=102, name='Charlie'}, Student{id=103, name='Bob'}]
- **Execution Explanation:** `Comparable` defines single natural ordering inside the domain class.

Problem 5: Custom Sorting with `Comparator` (26.4.5)

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

class Employee {
    int id;
    String name;
    double salary;

    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    @Override
    public String toString() {
        return name + " (" + salary + ")";
    }
}

public class ComparatorPracticeDemo {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();
        employees.add(new Employee(1, "John", 60000));
        employees.add(new Employee(2, "Alice", 85000));
        employees.add(new Employee(3, "Mark", 50000));

        // Sort descending by salary using Comparator lambda
        Collections.sort(employees, (e1, e2) -> Double.compare(e2.salary, e1.salary));

        System.out.println("Sorted by Salary Descending: " + employees);
    }
}

```

- **Expected Output:** Sorted by Salary Descending: [Alice (85000.0), John (60000.0), Mark (50000.0)]

- **Execution Explanation:** `Comparator` provides multiple, external custom sorting strategies without modifying the domain model.

2.2 Object-Oriented Programming (OOP) Coding Practice (Syllabus 26.5.1 to 26.5.7)

Problem 1: Encapsulation (26.5.1)

```

class BankAccount {
    private String accountNumber;
    private double balance;

    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        setBalance(initialBalance);
    }

    public String getAccountNumber() { return accountNumber; }
    public double getBalance() { return balance; }

    public void setBalance(double balance) {
        if (balance >= 0) {
            this.balance = balance;
        } else {
            System.out.println("Invalid balance amount!");
        }
    }
}

public class EncapsulationPracticeDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("ACC1001", 5000);
        acc.setBalance(-100); // Validation rejects invalid input
        System.out.println("Account: " + acc.getAccountNumber() + " | Balance: " + acc.getBalance());
    }
}

```

• **Expected Output:**

```

Invalid balance amount!
Account: ACC1001 | Balance: 5000.0

```

- **Execution Explanation:** Protects instance fields using `private` visibility and controlled validation logic in mutator methods.

Problem 2: Inheritance & Constructor Chaining (26.5.2)

```

class Parent {
    public Parent() {
        System.out.println("Parent No-Arg Constructor Executed");
    }
    public Parent(String msg) {
        System.out.println("Parent Parameterized Constructor: " + msg);
    }
}

class Child extends Parent {
    public Child() {
        super("Hello Super"); // Explicit super constructor invocation
        System.out.println("Child No-Arg Constructor Executed");
    }
}

public class InheritanceChainingDemo {
    public static void main(String[] args) {
        Child c = new Child();
    }
}

```

• **Expected Output:**

```

Parent Parameterized Constructor: Hello Super
Child No-Arg Constructor Executed

```

- **Execution Explanation:** Subclass constructors execute `super(...)` first to initialize inherited superclass state before subclass body execution.

Problem 3: Method Overloading (26.5.3)

```

public class OverloadingPracticeDemo {
    public int add(int a, int b) { return a + b; }
    public int add(int a, int b, int c) { return a + b + c; }
    public double add(double a, double b) { return a + b; }

    public static void main(String[] args) {
        OverloadingPracticeDemo demo = new OverloadingPracticeDemo();
        System.out.println("2 Ints: " + demo.add(5, 10));
        System.out.println("3 Ints: " + demo.add(5, 10, 15));
        System.out.println("2 Doubles: " + demo.add(2.5, 3.5));
    }
}

```

• **Expected Output:**

```

2 Ints: 15
3 Ints: 30
2 Doubles: 6.0

```

- **Execution Explanation:** Method overloading resolves function calls at compile-time based on argument signature matching.

Problem 4: Method Overriding (26.5.4)

```

class Animal {
    public void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Dog barks: Woof Woof!");
    }
}

public class OverridingPracticeDemo {
    public static void main(String[] args) {
        Animal myAnimal = new Dog(); // Parent reference to child object
        myAnimal.makeSound(); // Invokes overridden child method
    }
}

```

- **Expected Output:** Dog barks: Woof Woof!

- **Execution Explanation:** Dynamic Method Dispatch resolves overridden methods at runtime based on the actual object type in Heap memory.

Problem 5: Runtime Polymorphism (26.5.5)

```

abstract class Payment {
    abstract void processPayment(double amount);
}

class CreditCardPayment extends Payment {
    @Override
    void processPayment(double amount) {
        System.out.println("Processing Credit Card Payment of $" + amount);
    }
}

class UPIPayment extends Payment {
    @Override
    void processPayment(double amount) {
        System.out.println("Processing UPI Payment of $" + amount);
    }
}

public class PolymorphismPracticeDemo {
    public static void main(String[] args) {
        Payment p1 = new CreditCardPayment();
        Payment p2 = new UPIPayment();

        p1.processPayment(250.0);
        p2.processPayment(100.0);
    }
}

```

- **Expected Output:**

```

Processing Credit Card Payment of $250.0
Processing UPI Payment of $100.0

```

- **Execution Explanation:** Demonstrates loose coupling where superclass references handle diverse subclass implementations dynamically.

Problem 6: Abstract Class Mechanics (26.5.6)

```
abstract class Vehicle {
    private String brand;

    public Vehicle(String brand) {
        this.brand = brand;
    }

    public String getBrand() { return brand; }

    abstract void startEngine(); // Abstract method without body
}

class Car extends Vehicle {
    public Car(String brand) {
        super(brand);
    }

    @Override
    void startEngine() {
        System.out.println(getBrand() + " Car engine started with push button.");
    }
}

public class AbstractClassPracticeDemo {
    public static void main(String[] args) {
        Vehicle myCar = new Car("Toyota");
        myCar.startEngine();
    }
}
```

- **Expected Output:** `Toyota Car engine started with push button.`
- **Execution Explanation:** Abstract classes provide common concrete state/logic alongside mandatory abstract method contracts for subclasses.

Problem 7: Interface Implementation (26.5.7)

```

interface Printable {
    void print();
}

interface Showable {
    void show();
}

class Document implements Printable, Showable {
    @Override
    public void print() {
        System.out.println("Printing document...");
    }

    @Override
    public void show() {
        System.out.println("Showing document preview...");
    }
}

public class InterfacePracticeDemo {
    public static void main(String[] args) {
        Document doc = new Document();
        doc.print();
        doc.show();
    }
}

```

• **Expected Output:**

```

Printing document...
Showing document preview...

```

- **Execution Explanation:** Java supports multiple inheritance through interfaces without encountering state conflicts.

2.3 Exception Handling Coding Practice (Syllabus 26.6.1 to 26.6.4)

Problem 1: try-catch Exception Handling (26.6.1)

```

public class TryCatchPracticeDemo {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            int result = a / b; // Causes ArithmeticException
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Caught Exception: Division by zero is undefined!");
        }
        System.out.println("Application continues execution normally.");
    }
}

```

• **Expected Output:**

```

Caught Exception: Division by zero is undefined!
Application continues execution normally.

```

- **Execution Explanation:** Intercepts runtime arithmetic errors, preventing abrupt program termination.

Problem 2: Multiple Catch Blocks (26.6.2)

```
public class MultipleCatchPracticeDemo {
    public static void main(String[] args) {
        try {
            String str = null;
            System.out.println(str.length()); // Triggers NullPointerException
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception Caught");
        } catch (NullPointerException e) {
            System.out.println("NullPointerException Caught");
        } catch (Exception e) {
            System.out.println("General Exception Caught");
        }
    }
}
```

- **Expected Output:** `NullPointerException Caught`
- **Execution Explanation:** JVM matches exceptions against catch blocks in top-to-bottom order; specific subclasses must precede superclasses.

Problem 3: `throw` and `throws` Exception Propagation (26.6.3)

```
public class ThrowThrowsPracticeDemo {
    public static void validateAge(int age) throws IllegalArgumentException {
        if (age < 18) {
            throw new IllegalArgumentException("Access Denied: Age must be at least 18.");
        }
        System.out.println("Access Granted: Eligible for registration.");
    }

    public static void main(String[] args) {
        try {
            validateAge(15);
        } catch (IllegalArgumentException e) {
            System.out.println("Exception Handler: " + e.getMessage());
        }
    }
}
```

- **Expected Output:** `Exception Handler: Access Denied: Age must be at least 18.`
- **Execution Explanation:** `throw` explicitly raises an exception instance, while `throws` declares expected exception types in method signatures.

Problem 4: Custom User-Defined Exception (26.6.4)

```

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

public class CustomExceptionPracticeDemo {
    public static void withdraw(double balance, double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Withdrawal failed! Required: " + amount + ", Available: " + balance);
        }
        System.out.println("Withdrawal successful! Remaining: " + (balance - amount));
    }

    public static void main(String[] args) {
        try {
            withdraw(1000.0, 1500.0);
        } catch (InsufficientBalanceException e) {
            System.out.println("Custom Exception Caught: " + e.getMessage());
        }
    }
}

```

• **Expected Output:** Custom Exception Caught: Withdrawal failed! Required: 1500.0, Available: 1000.0

• **Execution Explanation:** Custom checked exceptions extend `java.lang.Exception` to enforce domain-specific business error handling.

CHAPTER 3 — CONCURRENCY, PERSISTENCE & SYSTEM-LEVEL CODING PRACTICE

3.1 Multithreading & Concurrency Coding Practice (Syllabus 26.7.1 to 26.7.4)

Problem 1: Thread Creation via `Thread` Class (26.7.1)

```

class MyCustomThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName() + " executing step: " + i);
            try { Thread.sleep(500); } catch (InterruptedException e) {}
        }
    }
}

public class ThreadClassPracticeDemo {
    public static void main(String[] args) {
        MyCustomThread t1 = new MyCustomThread();
        t1.setName("Worker-Thread-1");
        t1.start(); // Spawns new thread execution stack
    }
}

```

• **Expected Output:**

```
Worker-Thread-1 executing step: 1
Worker-Thread-1 executing step: 2
Worker-Thread-1 executing step: 3
```

- **Execution Explanation:** Extends `Thread` and overrides `run()`. Calling `start()` requests the OS scheduler to spawn a new call stack.

Problem 2: Thread Creation via `Runnable` Interface (26.7.2)

```
class MyRunnableTask implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable task executing inside: " + Thread.currentThread().getName());
    }
}

public class RunnablePracticeDemo {
    public static void main(String[] args) {
        MyRunnableTask task = new MyRunnableTask();
        Thread thread = new Thread(task, "Runnable-Worker");
        thread.start();
    }
}
```

- **Expected Output:** `Runnable task executing inside: Runnable-Worker`
- **Execution Explanation:** Implements `Runnable` interface, decoupling task execution logic from thread management.

Problem 3: Synchronization & Race Condition Prevention (26.7.3)

```

class SharedCounter {
    private int count = 0;

    // Synchronized method ensuring mutual exclusion
    public synchronized void increment() {
        count++;
    }

    public int getCount() { return count; }
}

public class SynchronizationPracticeDemo {
    public static void main(String[] args) throws InterruptedException {
        SharedCounter counter = new SharedCounter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        });

        t1.start();
        t2.start();

        t1.join(); // Wait for t1 completion
        t2.join(); // Wait for t2 completion

        System.out.println("Final Counter Value: " + counter.getCount());
    }
}

```

- **Expected Output:** Final Counter Value: 2000

- **Execution Explanation:** `synchronized` keyword enforces object-level locking, preventing race conditions during concurrent updates.

Problem 4: Inter-Thread Communication via `wait()` and `notify()` (26.7.4)

```

class SharedResource {
    private boolean available = false;

    public synchronized void produce() throws InterruptedException {
        while (available) {
            wait(); // Wait until consumer consumes
        }
        System.out.println("Produced item.");
        available = true;
        notify(); // Signal consumer thread
    }

    public synchronized void consume() throws InterruptedException {
        while (!available) {
            wait(); // Wait until producer produces
        }
        System.out.println("Consumed item.");
        available = false;
        notify(); // Signal producer thread
    }
}

public class WaitNotifyPracticeDemo {
    public static void main(String[] args) {
        SharedResource resource = new SharedResource();

        Thread producer = new Thread(() -> {
            try { resource.produce(); } catch (InterruptedException e) {}
        });

        Thread consumer = new Thread(() -> {
            try { resource.consume(); } catch (InterruptedException e) {}
        });

        consumer.start();
        producer.start();
    }
}

```

• **Expected Output:**

```

Produced item.
Consumed item.

```

- **Execution Explanation:** `wait()` releases intrinsic monitor locks and pauses threads; `notify()` awakens waiting threads safely.

3.2 JDBC & Database Connectivity Coding Practice (Syllabus 26.8.1 to 26.8.5)

Problem 1: JDBC Record Insertion (26.8.1)

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class JdbcInsertPracticeDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root", pass = "password";
        String sql = "INSERT INTO students (id, name, email) VALUES (?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(url, user, pass);
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, 101);
            pstmt.setString(2, "Alice");
            pstmt.setString(3, "alice@example.com");

            int rowsAffected = pstmt.executeUpdate();
            System.out.println("Inserted Rows: " + rowsAffected);
        } catch (SQLException e) {
            System.out.println("JDBC Error: " + e.getMessage());
        }
    }
}

```

- **Expected Output:** `Inserted Rows: 1`

- **Execution Explanation:** Binds positional parameter values safely to `PreparedStatement` and executes DML update.

Problem 2: JDBC SELECT Record Retrieval (26.8.2)

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class JdbcSelectPracticeDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root", pass = "password";
        String sql = "SELECT id, name, email FROM students";

        try (Connection conn = DriverManager.getConnection(url, user, pass);
            PreparedStatement pstmt = conn.prepareStatement(sql);
            ResultSet rs = pstmt.executeQuery()) {

            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                String email = rs.getString("email");
                System.out.println("ID: " + id + " | Name: " + name + " | Email: " + email);
            }
        } catch (SQLException e) {
            System.out.println("JDBC Error: " + e.getMessage());
        }
    }
}

```

- **Expected Output:** ID: 101 | Name: Alice | Email: alice@example.com

- **Execution Explanation:** `executeQuery()` returns tabular records inside a `ResultSet`; `rs.next()` navigates row by row.

Problem 3: JDBC UPDATE Operation (26.8.3)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class JdbcUpdatePracticeDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root", pass = "password";
        String sql = "UPDATE students SET email = ? WHERE id = ?";

        try (Connection conn = DriverManager.getConnection(url, user, pass);
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, "alice_updated@example.com");
            pstmt.setInt(2, 101);

            int rowsUpdated = pstmt.executeUpdate();
            System.out.println("Updated Rows: " + rowsUpdated);
        } catch (SQLException e) {
            System.out.println("JDBC Error: " + e.getMessage());
        }
    }
}
```

- **Expected Output:** Updated Rows: 1

- **Execution Explanation:** Parameterized `UPDATE` query updates existing table records safely.

Problem 4: JDBC DELETE Operation (26.8.4)

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class JdbcDeletePracticeDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root", pass = "password";
        String sql = "DELETE FROM students WHERE id = ?";

        try (Connection conn = DriverManager.getConnection(url, user, pass);
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, 101);

            int rowsDeleted = pstmt.executeUpdate();
            System.out.println("Deleted Rows: " + rowsDeleted);
        } catch (SQLException e) {
            System.out.println("JDBC Error: " + e.getMessage());
        }
    }
}

```

- **Expected Output:** Deleted Rows: 1
- **Execution Explanation:** Deletes targeted database table records based on primary key filter condition.

Problem 5: Complete End-to-End JDBC CRUD Application (26.8.5)

```

import java.sql.*;

public class CompleteCrudAppDemo {
    private static final String URL = "jdbc:mysql://localhost:3306/school_db";
    private static final String USER = "root";
    private static final String PASS = "password";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            System.out.println("--- 1. INSERT RECORD ---");
            try (PreparedStatement ins = conn.prepareStatement("INSERT INTO students (id, name) VALUE
S (?, ?)")) {
                ins.setInt(1, 201);
                ins.setString(2, "David");
                ins.executeUpdate();
            }

            System.out.println("--- 2. READ RECORD ---");
            try (PreparedStatement sel = conn.prepareStatement("SELECT * FROM students WHERE id = ?")
) {
                sel.setInt(1, 201);
                ResultSet rs = sel.executeQuery();
                if (rs.next()) System.out.println("Fetched: " + rs.getString("name"));
            }

            System.out.println("--- 3. UPDATE RECORD ---");
            try (PreparedStatement upd = conn.prepareStatement("UPDATE students SET name = ? WHERE id
= ?")) {
                upd.setString(1, "David Miller");
                upd.setInt(2, 201);
                upd.executeUpdate();
            }

            System.out.println("--- 4. DELETE RECORD ---");
            try (PreparedStatement del = conn.prepareStatement("DELETE FROM students WHERE id = ?"))
{
                del.setInt(1, 201);
                del.executeUpdate();
                System.out.println("Record deleted successfully.");
            }
        } catch (SQLException e) {
            System.out.println("CRUD Exception: " + e.getMessage());
        }
    }
}

```

• Expected Output:

```

--- 1. INSERT RECORD ---
--- 2. READ RECORD ---
Fetched: David
--- 3. UPDATE RECORD ---
--- 4. DELETE RECORD ---
Record deleted successfully.

```

- **Execution Explanation:** Combines Create, Read, Update, and Delete operations using JDBC [PreparedStatement](#) and [try-with-resources](#).

CHAPTER 4 — INTERVIEW DIAGNOSTICS, OUTPUT PREDICTION & DEBUGGING

4.1 Output Prediction Questions (Syllabus 26.9)

Note: The questions below are representative practice examples illustrating key output prediction patterns.

Question 1: String Pool vs Heap Object Comparison

```
public class OutputPred1 {
    public static void main(String[] args) {
        String s1 = "Java";
        String s2 = "Java";
        String s3 = new String("Java");
        String s4 = s3.intern();

        System.out.println(s1 == s2);
        System.out.println(s1 == s3);
        System.out.println(s1 == s4);
        System.out.println(s1.equals(s3));
    }
}
```

• **Output:**

```
true
false
true
true
```

- **Explanation:** `s1` and `s2` refer to the same object in the String Constant Pool (SCP). `s3` creates a distinct object in Heap memory, so `s1 == s3` is `false`. Calling `s3.intern()` returns the canonical SCP reference, so `s1 == s4` is `true`. `equals()` compares character values (`true`).

Question 2: Static vs Instance Initialization Order

```
class InitOrder {
    static { System.out.print("S1 "); }
    { System.out.print("I1 "); }
    public InitOrder() { System.out.print("C1 "); }
}

public class OutputPred2 {
    public static void main(String[] args) {
        System.out.print("M ");
        InitOrder obj1 = new InitOrder();
        InitOrder obj2 = new InitOrder();
    }
}
```

• **Output:** `S1 M I1 C1 I1 C1`

- **Explanation:** Class loading executes static blocks (`S1`) once. In `main()`, `M` is printed first. Each object instantiation executes the instance block (`I1`) followed by the constructor (`C1`).

Question 3: Exception Handling `finally` Return

```
public class OutputPred3 {
    public static int testMethod() {
        try {
            return 10;
        } catch (Exception e) {
            return 20;
        } finally {
            return 30; // Overrides preceding return statements
        }
    }

    public static void main(String[] args) {
        System.out.println("Returned: " + testMethod());
    }
}
```

- **Output:** Returned: 30
- **Explanation:** A `return` statement in a `finally` block discards any return value set in `try` or `catch` blocks.

4.2 Debugging & Tricky Code Analysis (Syllabus 26.10)

Note: The scenarios below are representative practice examples illustrating common debugging traps.

Bug 1: Modifying Captured Local Variable in Lambda

- **Flawed Code:**

```
int count = 0;
List<String> items = List.of("A", "B", "C");
items.forEach(item -> count++); // Compilation Error!
```

- **Cause:** Local variables captured by Lambdas must be `final` or effectively final.
- **Fix:** Use an array or atomic container:

```
int[] count = {0};
items.forEach(item -> count[0]++);
```

Bug 2: Missing Default Constructor in Inheritance

- **Flawed Code:**

```
class Parent {
    public Parent(int x) {}
}
class Child extends Parent {
    public Child() {} // Compilation Error!
}
```

- **Cause:** Defining a parameterized constructor suppresses the default no-arg constructor in `Parent`. `Child` constructor implicitly calls `super()`, which doesn't exist.
- **Fix:** Explicitly define a no-arg constructor in `Parent` or invoke `super(x)` inside `Child`.

4.3 Interview-Style Core Logic Questions (Syllabus 26.11.1 to 26.11.7)

Note: The items below (26.11.1 to 26.11.7) are representative practice examples covering core algorithmic logic.

26.11.1 Number Logic: Check if Number is Power of Two

```
public class PowerOfTwoDemo {
    public static boolean isPowerOfTwo(int n) {
        return n > 0 && (n & (n - 1)) == 0; // n > 0 and single set-bit check
    }

    public static void main(String[] args) {
        System.out.println("0 is Power of 2? " + isPowerOfTwo(0)); // false
        System.out.println("16 is Power of 2? " + isPowerOfTwo(16)); // true
        System.out.println("18 is Power of 2? " + isPowerOfTwo(18)); // false
    }
}
```

26.11.2 String Logic: Compress String Repeated Characters

```
public class StringCompressionDemo {
    public static String compress(String str) {
        StringBuilder sb = new StringBuilder();
        int count = 1;
        for (int i = 0; i < str.length(); i++) {
            if (i + 1 < str.length() && str.charAt(i) == str.charAt(i + 1)) {
                count++;
            } else {
                sb.append(str.charAt(i)).append(count);
                count = 1;
            }
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        System.out.println("Compressed 'aabcccccaaa': " + compress("aabcccccaaa")); // a2b1c5a3
    }
}
```

26.11.3 Array Logic: Move Zeros to End of Array

```
import java.util.Arrays;

public class MoveZerosDemo {
    public static void moveZeros(int[] nums) {
        int index = 0;
        for (int num : nums) {
            if (num != 0) nums[index++] = num;
        }
        while (index < nums.length) nums[index++] = 0;
    }

    public static void main(String[] args) {
        int[] nums = {0, 1, 0, 3, 12};
        moveZeros(nums);
        System.out.println("Zeros Moved: " + Arrays.toString(nums)); // [1, 3, 12, 0, 0]
    }
}
```

26.11.4 Collection Logic: Find Intersection of Two Lists

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class ListIntersectionDemo {
    public static void main(String[] args) {
        List<Integer> list1 = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
        List<Integer> list2 = Arrays.asList(3, 4, 5, 6, 7);

        list1.retainAll(list2); // Retains only elements present in list2
        System.out.println("Intersection: " + list1); // [3, 4, 5]
    }
}
```

26.11.5 OOP Logic: Prevent Class Inheritance and Method Overriding

- Declare class as **final** (`public final class FinalClass`) to prevent inheritance.
- Declare methods as **final** (`public final void doNotOverride()`) to prevent overriding.
- Make constructors **private** and provide static factory methods (Singleton pattern).

26.11.6 Exception Logic: Re-throwing Exceptions

```
public class RethrowExceptionDemo {
    public static void process() throws Exception {
        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Logging exception internally...");
            throw e; // Re-throwing exception to caller
        }
    }
}
```

26.11.7 JDBC Logic: Batch Execution for Performance

```
// Using addBatch() and executeBatch() to execute multiple statements efficiently
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO users VALUES (?, ?)");
for (User user : userList) {
    pstmt.setInt(1, user.getId());
    pstmt.setString(2, user.getName());
    pstmt.addBatch();
}
int[] updateCounts = pstmt.executeBatch(); // Executes single database round-trip
```

GRAND FINALE: FULL 8-BOOK SERIES MASTER PREPARATION & MOCK INTERVIEW PACK

1. COMPREHENSIVE MASTER REVISION SHEET (BOOKS 1 TO 8)

Core Fundamentals & OOP (Books 1–2)

- **Execution Pipeline:** `.java` $\rightarrow$ `javac` $\rightarrow$ `.class` $\rightarrow$ `ClassLoader` $\rightarrow$ `Bytecode Verifier` $\rightarrow$ `JIT/Interpreter` $\rightarrow$ `CPU`.
- **Memory Areas:** Stack (local variables, method frames), Heap (objects, instance variables), Metaspace/Method Area (class metadata, static variables, SCP).
- **OOP Pillars:** Encapsulation (data hiding via `private`), Inheritance (`extends`), Polymorphism (Overloading compile-time vs Overriding runtime), Abstraction (`abstract class`, `interface`).

Data Structures, Strings & Exceptions (Books 3–4)

- **Arrays & Strings:** Zero-indexed homogeneous memory; String immutability, SCP caching, `StringBuilder` vs `StringBuffer`.
- **Exception Hierarchy:** `Throwable` $\rightarrow$ `Error` (unrecoverable system faults) & `Exception` (Checked vs Runtime Unchecked).
- **Multithreading:** 6 official states (`NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`). `synchronized` enforces mutual exclusion.

Collections, I/O, Java 8 & JDBC (Books 5–7)

- **Collections:** `ArrayList` (resizable array), `LinkedList` (doubly-linked), `HashSet` (unordered $O(1)$), `TreeSet` (sorted $O(\log N)$), `HashMap` (buckets, separate chaining, Java 8 treeification).
- **Java 8:** Lambda expressions, Functional Interfaces (`Predicate`, `Function`, `Consumer`, `Supplier`), Stream API (`filter`, `map`, `collect`), `Optional`, `java.time`.
- **JDBC & Persistence:** `DriverManager` $\rightarrow$ `Connection` $\rightarrow$ `PreparedStatement` (SQL injection protection via `?`) $\rightarrow$ `ResultSet`.

2. TOP 50 FRESHER FULL STACK TECHNICAL INTERVIEW QUESTIONS

1. What is the difference between JDK, JRE, and JVM?
2. Explain Write Once, Run Anywhere (WORA).
3. What are primitive data types and their byte sizes in Java?
4. What is the difference between `==` and `.equals()`?
5. Why are String objects immutable in Java?
6. What is the String Constant Pool (SCP)?
7. What is the difference between `String`, `StringBuffer`, and `StringBuilder`?
8. Explain Method Overloading vs Method Overriding.
9. Can we override a `static` method? (No, it is method hiding).
10. What is the `this` keyword and constructor chaining?
11. What is the `super` keyword and `super()` invocation?

12. Why does Java not support multiple class inheritance?
13. What is Encapsulation and how is it achieved?
14. What is the difference between an `abstract class` and an `interface`?
15. What are default and static methods in Java 8 interfaces?
16. What is a Marker Interface? Give examples (`Serializable`, `Cloneable`).
17. What is the difference between `final`, `finally`, and `finalize()`?
18. What is the `equals()` and `hashCode()` contract?
19. What happens if you override `equals()` without `hashCode()`?
20. What is pass-by-value in Java?
21. What is the difference between Checked and Unchecked Exceptions?
22. How does `try-with-resources` work?
23. What is the difference between `throw` and `throws`?
24. What causes `StackOverflowError` and `OutOfMemoryError`?
25. Explain the two ways to create a thread in Java (`Thread` vs `Runnable`).
26. What is the difference between `start()` and `run()` methods?
27. What are the 6 official `Thread.State` enum values in Java?
28. What is Synchronization and why is it needed?
29. What is a Deadlock and how can it be prevented?
30. How do `wait()`, `notify()`, and `notifyAll()` work?
31. What is Autoboxing and Unboxing?
32. What is Generics and Type Erasure in Java?
33. What is the difference between `ArrayList` and `LinkedList`?
34. How does `HashMap` work internally in Java 8?
35. What is the default load factor and capacity of `HashMap`?
36. What is the difference between `HashSet` and `TreeSet`?
37. What is the difference between `Comparable` and `Comparator`?
38. What is `ConcurrentModificationException` and Fail-Fast iterators?
39. How does `CopyOnWriteArrayList` handle thread safety?
40. What is the difference between Byte Streams and Character Streams?
41. What is Serialization and the `transient` keyword?
42. What is a Lambda Expression?
43. What is a Functional Interface and `@FunctionalInterface`?
44. Name the 4 core built-in functional interfaces in `java.util.function`.
45. What is the Stream API and how does it differ from Collections?

46. What is the difference between Intermediate and Terminal stream operations?
47. How does `Optional` help handle null values?
48. What are the advantages of the `java.time` API over `java.util.Date`?
49. What are the 4 JDBC Driver Types?
50. Why is `PreparedStatement` preferred over `Statement`?
-

3. ULTIMATE FRESHER FULL STACK MOCK INTERVIEW SCENARIO

Interviewer: Welcome! Let's start with a foundational question. How does Java achieve platform independence?

Candidate: Thank you, sir! Java achieves platform independence through Bytecode and the Java Virtual Machine (JVM). When we compile `.java` source code using `javac`, it converts high-level code into intermediate `.class` bytecode rather than machine-specific native code. Any operating system with a JVM installed can translate this bytecode into native machine instructions at runtime. Thus: "Write Once, Run Anywhere."

Interviewer: Excellent! In Object-Oriented Programming, why should we prefer Interfaces over Abstract Classes for designing application contracts?

Candidate: Interfaces allow multiple inheritance of type, allowing a class to implement multiple interfaces (`class ServiceImpl implements AuthInterface, PaymentInterface`) without being restricted by Java's single class extension rule. Interfaces define pure contracts without holding state, enabling loose coupling and easy mocking during unit testing.

Interviewer: Perfect. How does `HashMap` resolve hash collisions in Java 8?

Candidate: In Java 8, `HashMap` uses separate chaining with singly-linked list nodes inside bucket array indices. When two keys produce the same bucket index, they are stored as linked nodes. If a single bucket's linked list grows beyond 8 elements and the total map capacity is at least 64, Java converts that linked list into a Red-Black Tree (`TreeNode`). This optimizes search time complexity from $O(N)$ linear time to $O(\log N)$ logarithmic time under heavy collisions.

Interviewer: Fantastic explanation! Finally, how does `PreparedStatement` protect database applications against SQL Injection?

Candidate: SQL Injection occurs when untrusted user input is directly concatenated into SQL query strings, allowing attackers to alter SQL execution logic. `PreparedStatement` uses parameterized templates with `?` placeholders. The database pre-compiles the query structure first. When input values are bound using `setString()` or `setInt()`, the database server treats them strictly as literal parameter data, never as executable SQL commands.

4. COMPLETE 8-BOOK SERIES SYLLABUS MASTER COVERAGE CHECKLIST

- [x] **Book 1: Java Fundamentals, Methods & Control Statements (Sections 1, 2, 3, 4)**
 - [x] Fundamentals, JVM/JRE/JDK, Memory Architecture, Operators
 - [x] Primitives, Scopes, Type Casting, Scanner Input
 - [x] Methods, Stack Frames, Overloading, Varargs, Recursion
 - [x] Conditional Statements, Loops, Jump Statements, 20 Solved Problems
- [x] **Book 2: OOP, Inheritance, Polymorphism, Abstraction & Interfaces (Sections 5, 8, 9, 10, 11, 12, 13, 14, 15, 16)**

- [x] Class, Object, Heap allocation, Constructors, [this](#), Pass-by-value
- [x] Static members, Execution Order, [public static void main](#) mechanics
- [x] Encapsulation, Private access, Getters/Setters, Access Modifiers
- [x] Inheritance, [super](#), Overriding, Dynamic Method Dispatch, [final](#)
- [x] Abstraction, Abstract Classes, Interfaces, Java 8 default/static methods, 18 Solved Problems
 - [x] **Book 3: Arrays, Strings & Object Class (Sections 6, 7, 17)**
- [x] 1D, 2D, Jagged Arrays, Searching, Sorting, 25 Solved Array Problems
- [x] String Pool (SCP), Immutability, [StringBuffer/StringBuilder](#), 20 Solved String Problems
- [x] [Object](#) class, [toString\(\)](#), [equals\(\)](#), [hashCode\(\)](#), Contract, 4 Solved Object Problems
 - [x] **Book 4: Exception Handling, Multithreading & Synchronization (Sections 18, 19, 20)**
- [x] Exceptions, Checked vs Unchecked, [try-catch-finally](#), [throw/throws](#), Custom Exceptions, 13 Solved Problems
- [x] Multithreading, [Thread](#) vs [Runnable](#), 6 Official Thread States, 8 Solved Problems
- [x] Synchronization, Locks, Deadlock, Inter-thread Communication ([wait/notify](#)), 6 Solved Problems
 - [x] **Book 5: Wrapper Classes, Generics & Collections (Sections 21, 22)**
- [x] Wrapper classes, Autoboxing/Unboxing, Generics, Type Erasure, 7 Solved Problems
- [x] Collections Framework: [ArrayList](#), [LinkedList](#), [HashSet](#), [TreeSet](#), [Queue](#), [PriorityQueue](#)
- [x] Hashing, [HashMap](#), [TreeMap](#), [Comparable](#), [Comparator](#), Fail-Fast iterators, 21 Solved Problems
 - [x] **Book 6: Java I/O & Java 8 Features (Sections 23, 24)**
- [x] File Handling, Byte vs Character streams, [BufferedReader/BufferedWriter](#), Serialization, [transient](#), 5 Solved Problems
- [x] Java 8 Lambda expressions, [@FunctionalInterface](#), Method references
- [x] Stream API ([filter](#), [map](#), [collect](#)), [Optional](#), [java.time](#) API, 6 Solved Problems
 - [x] **Book 7: JDBC, ORM, Hibernate & Maven (Section 25)**
- [x] Web Architecture, JDBC Driver Types (1–4), [Connection](#), [PreparedStatement](#), [ResultSet](#)
- [x] SQL Injection protection, Transactions ([commit/rollback](#)), [try-with-resources](#)
- [x] ORM concepts, Hibernate ORM architecture, JPA Annotations, Maven [pom.xml](#), 18 Solved Problems
 - [x] **Book 8: Complete Coding Practice & Interview Preparation (Section 26)**
- [x] Number, String, and Array-based Algorithmic Practice Problems (26.1 – 26.3)
- [x] Collections, OOP, and Exception Handling Practice Problems (26.4 – 26.6)
- [x] Multithreading and JDBC Practice Problems (26.7 – 26.8)
- [x] Output Prediction, Debugging & Interview Logic Challenges (26.9 – 26.11)
- [x] Complete 8-Book Master Revision Sheet, Top 50 Interview Q&A, Mock Interview, Master Checklist

CONGRATULATIONS!

YOU HAVE COMPLETED THE COMPLETE 8-BOOK JAVA FULL STACK TEXTBOOK SERIES!